Exploratory Data Analysis and Preprocessing

This section loads the dataset, profiles its structure, identifies and resolves data quality issues (missing values, invalid entries, outliers), explores relationships between features and the target variable, and prepares the data for modeling (encoding, scaling, train/test split). Our target variable, `ev_adoption_likelihood`, is an ordinal categorical variable with three classes: Low, Medium, and High.

```
import pandas as pd
```

```
from google.colab import files

uploaded = files.upload()
```

```
Choose Files No file chosen  Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.
Saving global_ev_adoption_behavior_2026.csv to global_ev_adoption_behavior_2026.csv
```

```
df = pd.read_csv("global_ev_adoption_behavior_2026.csv")
```

Initial Data Profiling

We start by confirming the dataset's structure and size (50,000 rows, 23 columns), reviewing column data types, and checking for any immediate structural issues before diving into data quality.

```
df.head()
```

	age	annual_income	education_level	city_type	daily_commute_km	weekly_travel_distance_km	current_vehicle_type	vehicle_age_years	fuel_expense_per_month	c
0	56	23019.0	High School	Suburban	39.8	205.7	Hatchback	6.1	317.1	
1	46	26440.0	High School	Suburban	34.6	218.4	Sedan	4.4	290.0	
2	46	57167.0	PhD	Suburban	30.5	177.7	Sedan	0.4	201.4	
3	23	15841.0	Master	Suburban	44.6	325.9	SUV	0.0	407.3	
4	50	51571.0	Master	Urban	52.4	281.0	SUV	5.2	458.4	

5 rows × 23 columns

```
df.shape
```

```
(50000, 23)
```

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 50000 entries, 0 to 49999
Data columns (total 23 columns):
 #   Column                          Non-Null Count  Dtype
---  ------                          --------------  -----
```

```
 0   age                            50000 non-null  int64
 1   annual_income                  50000 non-null  float64
 2   education_level                49500 non-null  object
 3   city_type                      50000 non-null  object
 4   daily_commute_km               50000 non-null  float64
 5   weekly_travel_distance_km      50000 non-null  float64
 6   current_vehicle_type           50000 non-null  object
 7   vehicle_age_years              50000 non-null  float64
 8   fuel_expense_per_month         50000 non-null  float64
 9   charging_station_accessibility 49500 non-null  float64
 10  nearest_charging_station_km    50000 non-null  float64
 11  home_charging_available        50000 non-null  int64
 12  electricity_cost_per_kwh       50000 non-null  float64
 13  environmental_awareness_score  50000 non-null  float64
 14  government_incentive_awareness 50000 non-null  float64
 15  technology_affinity_score      50000 non-null  float64
 16  range_anxiety_score            50000 non-null  float64
 17  battery_replacement_concern    50000 non-null  float64
 18  ev_knowledge_score             49500 non-null  float64
 19  previous_ev_experience         50000 non-null  int64
 20  ev_adoption_likelihood         50000 non-null  object
 21  monthly_energy_consumption_kwh 50000 non-null  float64
 22  monthly_charging_cost          50000 non-null  float64
dtypes: float64(16), int64(3), object(4)
memory usage: 8.8+ MB
```

```
df.dtypes
```

	0
age	int64
annual_income	float64
education_level	object
city_type	object
daily_commute_km	float64
weekly_travel_distance_km	float64
current_vehicle_type	object
vehicle_age_years	float64
fuel_expense_per_month	float64
charging_station_accessibility	float64
nearest_charging_station_km	float64
home_charging_available	int64
electricity_cost_per_kwh	float64
environmental_awareness_score	float64
government_incentive_awareness	float64
technology_affinity_score	float64
range_anxiety_score	float64
battery_replacement_concern	float64
ev_knowledge_score	float64
previous_ev_experience	int64
ev_adoption_likelihood	object
monthly_energy_consumption_kwh	float64
monthly_charging_cost	float64

dtype: object

Missing Value Check

Three columns have missing values: `education_level`, `charging_station_accessibility`, and `ev_knowledge_score`, each at ~1% (500 rows). Given the low proportion, we address these later with simple imputation rather than dropping rows, to avoid unnecessary data loss.

```
missing_summary = pd.DataFrame({
    "Missing Values": df.isnull().sum(),
    "Missing Percentage": (df.isnull().sum() / len(df)) * 100
})

missing_summary[missing_summary["Missing Values"] > 0]
```

	Missing Values	Missing Percentage
education_level	500	1.0
charging_station_accessibility	500	1.0
ev_knowledge_score	500	1.0

Duplicate Check

No duplicate rows were found, so no deduplication is needed.

```
df.duplicated().sum()
```

```
np.int64(0)
```

Target Variable Distribution

The target class is imbalanced: ~59% High, ~24% Medium, ~17% Low. We account for this later by stratifying the train/test split so both sets preserve these proportions.

`df.describe()` also surfaces a red flag: `fuel_expense_per_month` has a minimum of -99.70, which is not physically possible and needs correction.

```
df["ev_adoption_likelihood"].value_counts()
```

ev_adoption_likelihood	count
High	29670
Medium	12078
Low	8252

dtype: int64

```
df["ev_adoption_likelihood"].value_counts(normalize=True) * 100
```

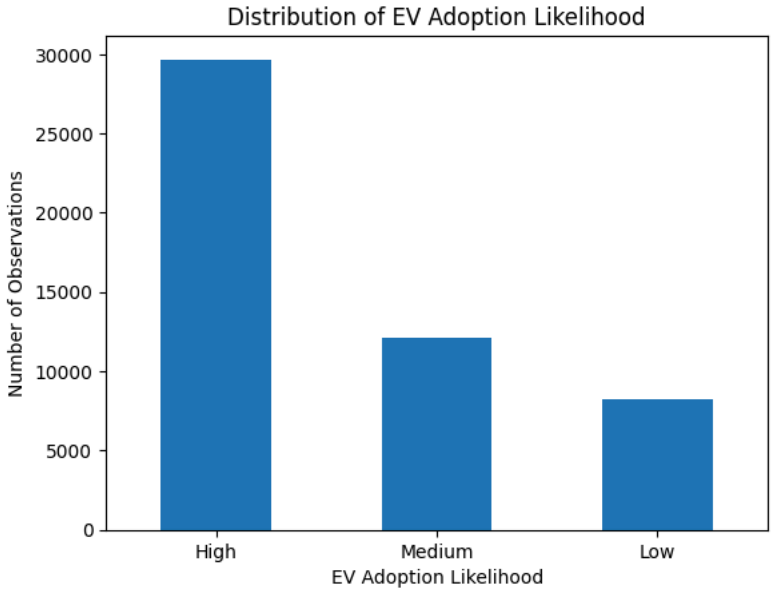
ev_adoption_likelihood	proportion
High	59.340
Medium	24.156
Low	16.504

dtype: float64

```
import matplotlib.pyplot as plt
```

```
df["ev_adoption_likelihood"].value_counts().plot(kind="bar")

plt.title("Distribution of EV Adoption Likelihood")
plt.xlabel("EV Adoption Likelihood")
plt.ylabel("Number of Observations")
plt.xticks(rotation=0)
plt.show()
```




```
df.describe().T
```

	count	mean	std	min	25%	50%	75%	max
age	50000.0	44.996080	14.110844	21.00	33.00	45.00	57.00	69.00
annual_income	50000.0	44311.869380	26109.170784	5104.00	25617.75	37891.00	56051.25	250000.00
daily_commute_km	50000.0	35.169358	14.736042	5.00	25.00	35.10	45.20	97.90
weekly_travel_distance_km	50000.0	228.727200	101.147254	25.00	157.70	224.10	294.50	732.30
vehicle_age_years	50000.0	6.012842	2.947132	0.00	3.90	6.00	8.00	20.00
fuel_expense_per_month	50000.0	295.169608	130.178552	-99.70	205.20	294.80	383.00	862.00
charging_station_accessibility	49500.0	5.892947	2.152186	1.00	4.40	5.90	7.40	10.00
nearest_charging_station_km	50000.0	7.132370	4.801992	0.50	3.00	6.90	10.70	28.50
home_charging_available	50000.0	0.649760	0.477050	0.00	0.00	1.00	1.00	1.00
electricity_cost_per_kwh	50000.0	0.215359	0.077914	0.08	0.15	0.22	0.28	0.35
environmental_awareness_score	50000.0	6.831854	1.920217	2.00	5.40	7.00	8.40	10.00
government_incentive_awareness	50000.0	5.979970	1.955106	1.00	4.60	6.00	7.30	10.00
technology_affinity_score	50000.0	6.721914	1.990730	2.00	5.20	6.70	8.50	10.00
range_anxiety_score	50000.0	5.288046	2.059862	2.00	3.70	5.00	6.80	10.00
battery_replacement_concern	50000.0	4.998338	1.955411	1.00	3.60	5.00	6.30	10.00
ev_knowledge_score	49500.0	6.755673	1.945842	1.00	5.30	7.00	8.30	10.00
previous_ev_experience	50000.0	0.198200	0.398648	0.00	0.00	0.00	0.00	1.00
monthly_energy_consumption_kwh	50000.0	186.740260	88.245302	15.60	124.50	179.30	241.50	645.90
monthly_charging_cost	50000.0	40.235028	24.940343	1.40	21.60	35.20	54.10	199.60

Data Error Correction and Imputation

Negative fuel expense: 271 rows have negative `fuel_expense_per_month` values, which cannot occur in reality. We treat this as a data entry error (likely a sign error) and correct it using absolute value rather than dropping the rows, since the magnitude still appears valid and 271 rows is a small fraction of the data (~0.5%).

Missing values: `education_level` is imputed with its mode (most common category), since it's categorical. `charging_station_accessibility` and `ev_knowledge_score` are imputed with their medians, since both are bounded 1–10 score variables where median is more robust to skew than mean.

```
(df["fuel_expense_per_month"] < 0).sum()
```

```
np.int64(271)
```

```
df[df["fuel_expense_per_month"] < 0][
    ["fuel_expense_per_month", "daily_commute_km", "current_vehicle_type"]
].head(10)
```

	fuel_expense_per_month	daily_commute_km	current_vehicle_type
18	-0.7	5.0	Sedan
27	-0.8	7.9	Truck
29	-19.2	5.0	Sedan
137	-11.9	6.0	Sedan
492	-4.5	5.7	Hatchback
688	-22.5	9.7	Sedan
746	-9.2	5.0	SUV
821	-17.5	5.1	Hatchback
874	-20.8	5.0	Sedan
1089	-8.4	5.0	Sedan

```
# Fix negative fuel_expense_per_month values using absolute values
df["fuel_expense_per_month"] = df["fuel_expense_per_month"].abs()

# verify removal of negative values
print((df["fuel_expense_per_month"] < 0).sum())

# Impute missing values
df["education_level"] = df["education_level"].fillna(df["education_level"].mode()[0])

df["charging_station_accessibility"] = df["charging_station_accessibility"].fillna(
    df["charging_station_accessibility"].median()
)

df["ev_knowledge_score"] = df["ev_knowledge_score"].fillna(
    df["ev_knowledge_score"].median()
)

# verify no missing values remain
print(df.isnull().sum().sum())
```

```
0
0
```

Outlier Detection

Using the IQR method (1.5×IQR beyond Q1/Q3) across the remaining numeric columns, outlier rates range from 0% (`age`, `electricity_cost_per_kwh`) up to ~3.8% (`annual_income`). All flagged outliers are right-tail only (no implausible lower-bound values), consistent with natural right-skew in variables like income and travel distance rather than data errors. We retain these values rather than removing or capping them, since they represent plausible real-world cases. We note `annual_income` and `monthly_charging_cost` as candidates for log-transformation if a linear/distance-based model proves sensitive to their skew.

```
import numpy as np

# Numeric columns to check for outliers
```

```
cols_to_check = [
    "age", "annual_income", "daily_commute_km", "weekly_travel_distance_km",
    "vehicle_age_years", "fuel_expense_per_month", "nearest_charging_station_km",
    "electricity_cost_per_kwh", "monthly_energy_consumption_kwh", "monthly_charging_cost"
]

# IQR method - flag values outside 1.5*IQR from Q1/Q3
outlier_summary = {}

for col in cols_to_check:
    Q1 = df[col].quantile(0.25)
    Q3 = df[col].quantile(0.75)
    IQR = Q3 - Q1
    lower = Q1 - 1.5 * IQR
    upper = Q3 + 1.5 * IQR
    outliers = df[(df[col] < lower) | (df[col] > upper)]
    outlier_summary[col] = len(outliers)

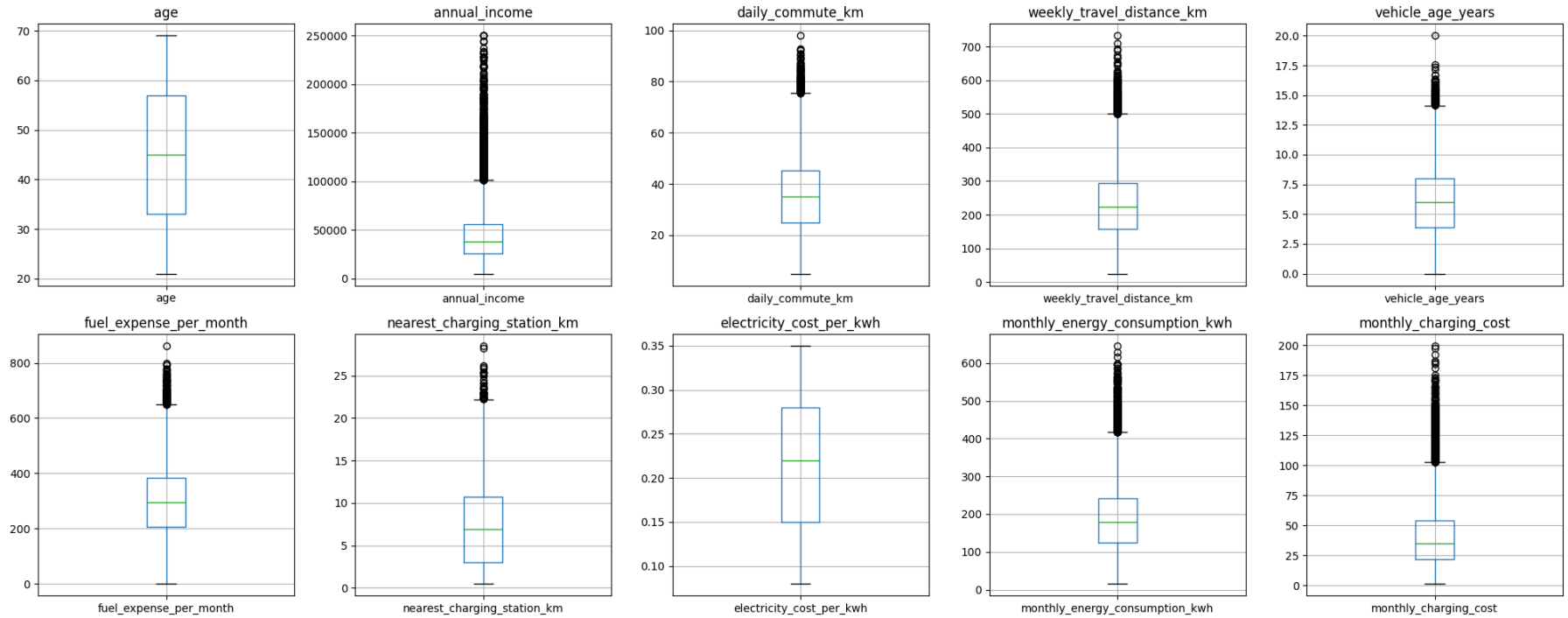
outlier_df = pd.DataFrame.from_dict(outlier_summary, orient="index", columns=["outlier_count"])
outlier_df["outlier_pct"] = (outlier_df["outlier_count"] / len(df)) * 100
print(outlier_df.sort_values("outlier_pct", ascending=False))
```

```
                               outlier_count  outlier_pct
annual_income                           1882        3.764
monthly_charging_cost                   1124        2.248
monthly_energy_consumption_kwh           646        1.292
weekly_travel_distance_km                377        0.754
daily_commute_km                         192        0.384
fuel_expense_per_month                   173        0.346
vehicle_age_years                        170        0.340
nearest_charging_station_km               42        0.084
age                                        0        0.000
electricity_cost_per_kwh                   0        0.000
```

```
# Boxplots
fig, axes = plt.subplots(2, 5, figsize=(20, 8))
axes = axes.flatten()

for i, col in enumerate(cols_to_check):
    df.boxplot(column=col, ax=axes[i])
    axes[i].set_title(col)

plt.tight_layout()
plt.show()
```



Correlation Analysis

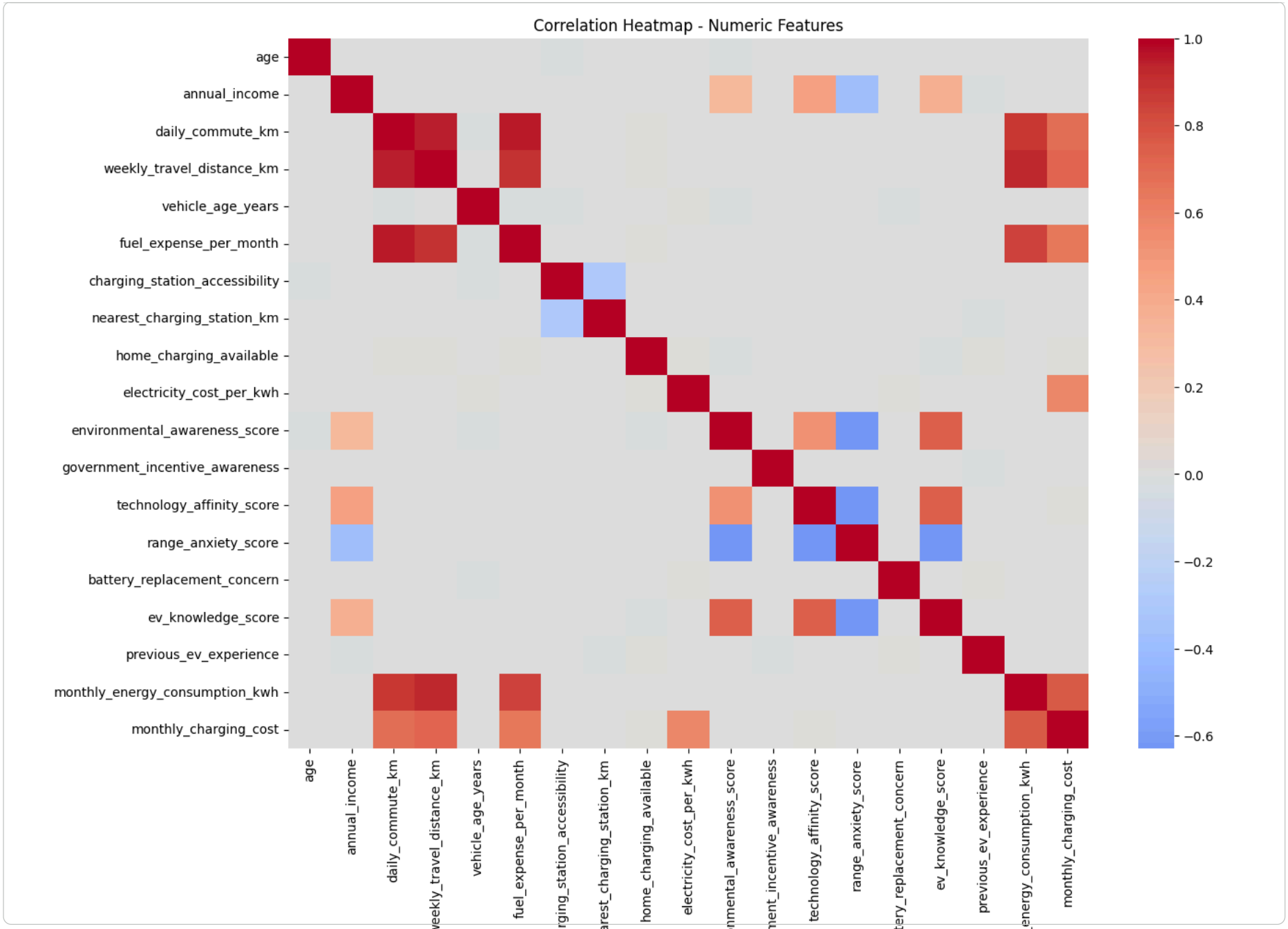
A correlation heatmap across numeric features reveals a multicollinear cluster: `daily_commute_km`, `weekly_travel_distance_km`, `fuel_expense_per_month`, `monthly_energy_consumption_kwh`, and `monthly_charging_cost` are all correlated at 0.7–0.95, which makes intuitive sense (more driving drives up fuel/energy/charging costs together). We address this below by dropping two redundant columns.

```
# Correlation heatmap (numeric features only)
import seaborn as sns

numeric_cols = df.select_dtypes(include=[np.number]).columns
corr = df[numeric_cols].corr()
```

```
plt.figure(figsize=(14, 10))
sns.heatmap(corr, cmap="coolwarm", center=0, annot=False)
plt.title("Correlation Heatmap - Numeric Features")
plt.show()

# Top 10 Correlated features
corr_pairs = corr.abs().unstack().sort_values(ascending=False)
corr_pairs = corr_pairs[corr_pairs < 1].drop_duplicates()
print(corr_pairs.head(10))
```



Feature Relationships with Target

```
daily_commute_km              fuel_expense_per_month        0.951789
weekly_travel_distance_km     daily_commute_km              0.944852
```

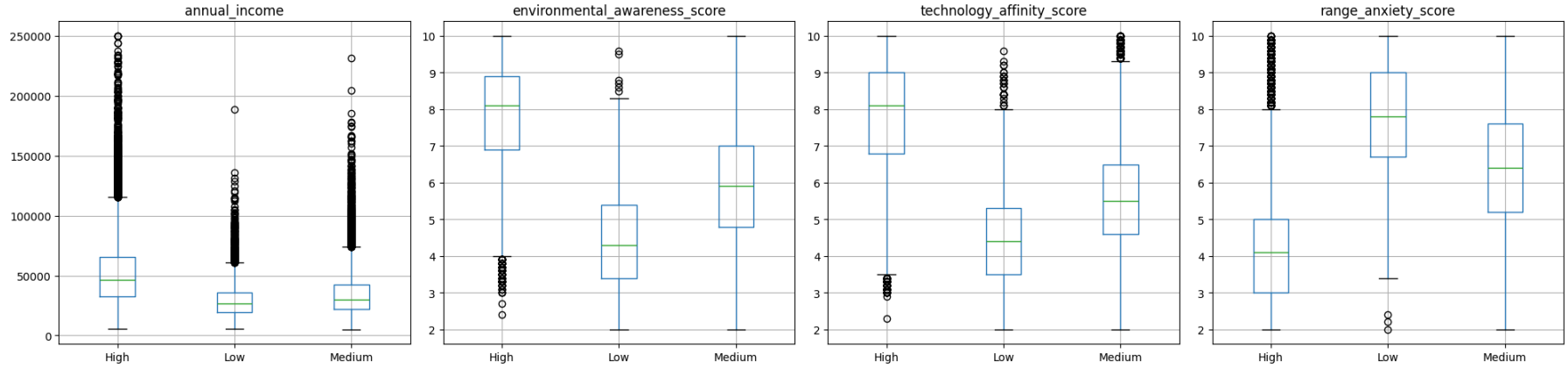
```
monthly_energy_consumption_kwh  weekly_travel_distance
weekly_travel_distance_km       fuel_expense_per_month
fuel_expense_per_month          monthly_energy_consumption_kwh  0.842395
monthly_energy_consumption_kwh  monthly_charging_cost           0.765091
ev_knowledge_score              technology_affinity_score       0.752686
```

Grouping key features by adoption class shows clean separation for `environmental_awareness_score`, `technology_affinity_score`, and `range_anxiety_score` — High-adoption respondents show higher awareness/tech affinity and lower range anxiety than Low-adoption respondents. These are expected to be strong predictors for modeling.

```
# Boxplots of a few key features grouped by target class
features_to_compare = ["annual_income", "environmental_awareness_score",
                       "technology_affinity_score", "range_anxiety_score"]

fig, axes = plt.subplots(1, 4, figsize=(20, 5))
for i, col in enumerate(features_to_compare):
    df.boxplot(column=col, by="ev_adoption_likelihood", ax=axes[i])
    axes[i].set_title(col)
    axes[i].set_xlabel("")

plt.suptitle("")
plt.tight_layout()
plt.show()
```



Feature Selection: Resolving Multicollinearity

To reduce redundancy identified in the correlation analysis, we drop `daily_commute_km` (redundant with `weekly_travel_distance_km`, which captures the same usage pattern at a less noisy weekly granularity) and `monthly_energy_consumption_kwh` (redundant with `monthly_charging_cost`, the more directly interpretable variable). This reduces multicollinearity for linear models without losing meaningful insight.

```
# Drop redundant/multicollinear columns
df = df.drop(columns=["daily_commute_km", "monthly_energy_consumption_kwh"])
```

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import classification_report, accuracy_score
```

```
df = pd.read_csv("/content/global_ev_adoption_behavior_2026.csv")
df.head()
```

	age	annual_income	education_level	city_type	daily_commute_km	weekly_travel_distance_km	current_vehicle_type	vehicle_age_years	fuel_expense_per_month	c
0	56	23019.0	High School	Suburban	39.8	205.7	Hatchback	6.1	317.1	
1	46	26440.0	High School	Suburban	34.6	218.4	Sedan	4.4	290.0	
2	46	57167.0	PhD	Suburban	30.5	177.7	Sedan	0.4	201.4	
3	23	15841.0	Master	Suburban	44.6	325.9	SUV	0.0	407.3	
4	50	51571.0	Master	Urban	52.4	281.0	SUV	5.2	458.4	

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 50000 entries, 0 to 49999
Data columns (total 23 columns):
 #   Column                          Non-Null Count  Dtype
---  ------                          --------------  -----
 0   age                             50000 non-null  int64
 1   annual_income                   50000 non-null  float64
 2   education_level                 49500 non-null  object
 3   city_type                       50000 non-null  object
 4   daily_commute_km                50000 non-null  float64
 5   weekly_travel_distance_km       50000 non-null  float64
 6   current_vehicle_type            50000 non-null  object
 7   vehicle_age_years               50000 non-null  float64
 8   fuel_expense_per_month          50000 non-null  float64
 9   charging_station_accessibility  49500 non-null  float64
 10  nearest_charging_station_km     50000 non-null  float64
 11  home_charging_available         50000 non-null  int64
 12  electricity_cost_per_kwh        50000 non-null  float64
 13  environmental_awareness_score   50000 non-null  float64
 14  government_incentive_awareness  50000 non-null  float64
 15  technology_affinity_score       50000 non-null  float64
 16  range_anxiety_score             50000 non-null  float64
 17  battery_replacement_concern     50000 non-null  float64
 18  ev_knowledge_score              49500 non-null  float64
 19  previous_ev_experience          50000 non-null  int64
 20  ev_adoption_likelihood          50000 non-null  object
 21  monthly_energy_consumption_kwh  50000 non-null  float64
 22  monthly_charging_cost           50000 non-null  float64
dtypes: float64(16), int64(3), object(4)
memory usage: 8.8+ MB
```

```
# Encode categorical text features to numeric integers
label_encoders = {}
```

```
for col in df.select_dtypes(include=['object']).columns:
    # Skip processing your target column here if it is an object
    if col != 'ev_adoption_likelihood':
        le = LabelEncoder()
        df[col] = le.fit_transform(df[col].astype(str))
        label_encoders[col] = le
```

```
X = df.drop(columns=['ev_adoption_likelihood'])
y = df['ev_adoption_likelihood']
```

```
# 4. Split into a training set (80%) and a testing set (20%)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# 5. Scale features (mandatory step for distance-based models like KNN)
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

```
from sklearn.impute import SimpleImputer
import numpy as np

# Impute missing values in X_train_scaled and X_test_scaled
imputer = SimpleImputer(missing_values=np.nan, strategy='mean')
X_train_scaled = imputer.fit_transform(X_train_scaled)
X_test_scaled = imputer.transform(X_test_scaled)

# 6. Initialize and fit the K-NN Classifier
knn = KNeighborsClassifier(n_neighbors=3)
knn.fit(X_train_scaled, y_train)

# 7. Make predictions on the test data
y_pred = knn.predict(X_test_scaled)

# 8. View performance metrics
print(f"Test Set Accuracy: {accuracy_score(y_test, y_pred):.4f}")
print("\nClassification Report:\n")
print(classification_report(y_test, y_pred))
```

```
Test Set Accuracy: 0.7925

Classification Report:

              precision    recall  f1-score   support

        High       0.87      0.93      0.90      5934
         Low       0.76      0.67      0.71      1650
      Medium       0.59      0.54      0.56      2416

    accuracy                           0.79     10000
   macro avg       0.74      0.71      0.73     10000
weighted avg       0.78      0.79      0.79     10000
```

```
from sklearn.model_selection import cross_val_score
from sklearn.neighbors import KNeighborsClassifier

k_values = [3, 5, 7, 9, 11, 15, 20, 25]
cv_scores = []

for k in k_values:
    knn = KNeighborsClassifier(n_neighbors=k)
    scores = cross_val_score(knn, X_train_scaled, y_train, cv=5, scoring='accuracy')
    cv_scores.append(scores.mean())

for k, score in zip(k_values, cv_scores):
    print(f"k={k}: CV Accuracy = {score:.4f}")

best_k = k_values[cv_scores.index(max(cv_scores))]
print(f"Best k value: {best_k}")
```

```
k=3: CV Accuracy = 0.7886
k=5: CV Accuracy = 0.7969
k=7: CV Accuracy = 0.8095
k=9: CV Accuracy = 0.8144
k=11: CV Accuracy = 0.8196
k=15: CV Accuracy = 0.8256
k=20: CV Accuracy = 0.8306
k=25: CV Accuracy = 0.8316
Best k value: 25
```

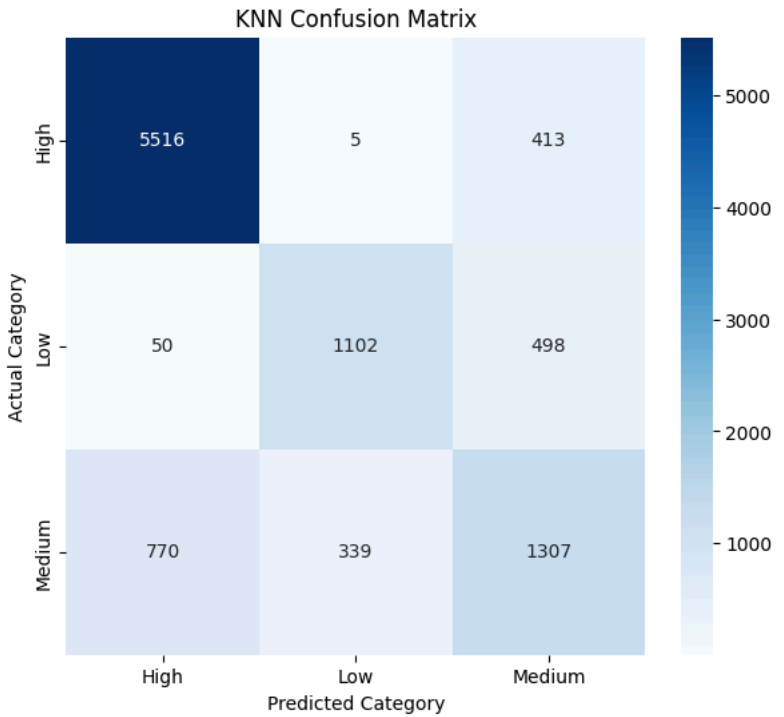
```
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.metrics import confusion_matrix

# Generate the confusion matrix array
cm = confusion_matrix(y_test, y_pred)

# Define the class labels based on the presumed encoding (0: High, 1: Low, 2: Medium)
class_labels = ['High', 'Low', 'Medium']

# Plot using Seaborn's heatmap
plt.figure(figsize=(7, 6)) # Adjusted figure size for better readability
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=class_labels,
            yticklabels=class_labels)

plt.ylabel('Actual Category')
plt.xlabel('Predicted Category')
plt.title('KNN Confusion Matrix')
plt.show()
```




```
from sklearn.metrics import precision_recall_curve, average_precision_score
from sklearn.preprocessing import label_binarize

# Get probability estimates for all classes
y_proba = knn.predict_proba(X_test_scaled)

# For multi-class, plot one Precision-Recall curve per class (e.g., for 'High' adoption, which is class 0)
# Binarize the true labels for class 0
y_test_binarized_class_0 = (y_test == 0).astype(int)

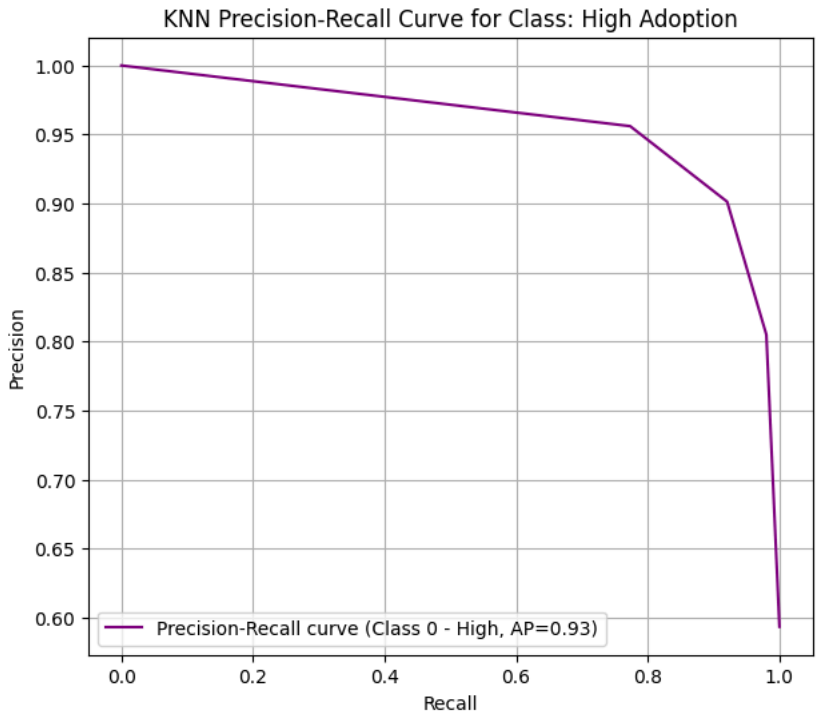
# Get the probability estimates for class 0
y_probs_class_0 = y_proba[:, 0]

# Calculate precision and recall for class 0
precision_class_0, recall_class_0, _ = precision_recall_curve(y_test_binarized_class_0, y_probs_class_0)

# Calculate Average Precision Score for class 0
avg_precision_class_0 = average_precision_score(y_test_binarized_class_0, y_probs_class_0)

plt.figure(figsize=(7, 6))
plt.plot(recall_class_0, precision_class_0, color='purple', label=f'Precision-Recall curve (Class 0 - High, AP={avg_precision_class_0:.2f})')
plt.xlabel('Recall')
plt.ylabel('Precision')
plt.title('KNN Precision-Recall Curve for Class: High Adoption')
plt.legend(loc='lower left')
plt.grid(True)
plt.show()
```

```
# You can extend this to plot for other classes (1 and 2) similarly
```



Exploratory Data Analysis and Preprocessing

This section loads the dataset, profiles its structure, identifies and resolves data quality issues (missing values, invalid entries, outliers), explores relationships between features and the target variable, and prepares the data for modeling (encoding, scaling, train/test split). Our target variable, `ev_adoption_likelihood`, is an ordinal categorical variable with three classes: Low, Medium, and High.

```
import pandas as pd
```

```
from google.colab import files

uploaded = files.upload()
```

```
Choose Files No file chosen  Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.
Saving global_ev_adoption_behavior_2026.csv to global_ev_adoption_behavior_2026.csv
```

```
df = pd.read_csv("global_ev_adoption_behavior_2026.csv")
```

Initial Data Profiling

We start by confirming the dataset's structure and size (50,000 rows, 23 columns), reviewing column data types, and checking for any immediate structural issues before diving into data quality.

```
df.head()
```

	age	annual_income	education_level	city_type	daily_commute_km	weekly_travel_distance_km	current_vehicle_type	vehicle_age_years	fuel_expense_per_month	c
0	56	23019.0	High School	Suburban	39.8	205.7	Hatchback	6.1	317.1	
1	46	26440.0	High School	Suburban	34.6	218.4	Sedan	4.4	290.0	
2	46	57167.0	PhD	Suburban	30.5	177.7	Sedan	0.4	201.4	
3	23	15841.0	Master	Suburban	44.6	325.9	SUV	0.0	407.3	
4	50	51571.0	Master	Urban	52.4	281.0	SUV	5.2	458.4	

5 rows × 23 columns

```
df.shape
```

```
(50000, 23)
```

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 50000 entries, 0 to 49999
Data columns (total 23 columns):
 #   Column                          Non-Null Count  Dtype
---  ------                          --------------  -----
```

```
 0   age                            50000 non-null  int64
 1   annual_income                  50000 non-null  float64
 2   education_level                49500 non-null  object
 3   city_type                      50000 non-null  object
 4   daily_commute_km               50000 non-null  float64
 5   weekly_travel_distance_km      50000 non-null  float64
 6   current_vehicle_type           50000 non-null  object
 7   vehicle_age_years              50000 non-null  float64
 8   fuel_expense_per_month         50000 non-null  float64
 9   charging_station_accessibility 49500 non-null  float64
 10  nearest_charging_station_km    50000 non-null  float64
 11  home_charging_available        50000 non-null  int64
 12  electricity_cost_per_kwh       50000 non-null  float64
 13  environmental_awareness_score  50000 non-null  float64
 14  government_incentive_awareness 50000 non-null  float64
 15  technology_affinity_score      50000 non-null  float64
 16  range_anxiety_score            50000 non-null  float64
 17  battery_replacement_concern    50000 non-null  float64
 18  ev_knowledge_score             49500 non-null  float64
 19  previous_ev_experience         50000 non-null  int64
 20  ev_adoption_likelihood         50000 non-null  object
 21  monthly_energy_consumption_kwh 50000 non-null  float64
 22  monthly_charging_cost          50000 non-null  float64
dtypes: float64(16), int64(3), object(4)
memory usage: 8.8+ MB
```

```
df.dtypes
```

	0
age	int64
annual_income	float64
education_level	object
city_type	object
daily_commute_km	float64
weekly_travel_distance_km	float64
current_vehicle_type	object
vehicle_age_years	float64
fuel_expense_per_month	float64
charging_station_accessibility	float64
nearest_charging_station_km	float64
home_charging_available	int64
electricity_cost_per_kwh	float64
environmental_awareness_score	float64
government_incentive_awareness	float64
technology_affinity_score	float64
range_anxiety_score	float64
battery_replacement_concern	float64
ev_knowledge_score	float64
previous_ev_experience	int64
ev_adoption_likelihood	object
monthly_energy_consumption_kwh	float64
monthly_charging_cost	float64

dtype: object

Missing Value Check

Three columns have missing values: `education_level`, `charging_station_accessibility`, and `ev_knowledge_score`, each at ~1% (500 rows). Given the low proportion, we address these later with simple imputation rather than dropping rows, to avoid unnecessary data loss.

```
missing_summary = pd.DataFrame({
    "Missing Values": df.isnull().sum(),
    "Missing Percentage": (df.isnull().sum() / len(df)) * 100
})

missing_summary[missing_summary["Missing Values"] > 0]
```

	Missing Values	Missing Percentage
education_level	500	1.0
charging_station_accessibility	500	1.0
ev_knowledge_score	500	1.0

Duplicate Check

No duplicate rows were found, so no deduplication is needed.

```
df.duplicated().sum()
```

```
np.int64(0)
```

Target Variable Distribution

The target class is imbalanced: ~59% High, ~24% Medium, ~17% Low. We account for this later by stratifying the train/test split so both sets preserve these proportions.

`df.describe()` also surfaces a red flag: `fuel_expense_per_month` has a minimum of -99.70, which is not physically possible and needs correction.

```
df["ev_adoption_likelihood"].value_counts()
```

ev_adoption_likelihood	count
High	29670
Medium	12078
Low	8252

dtype: int64

```
df["ev_adoption_likelihood"].value_counts(normalize=True) * 100
```

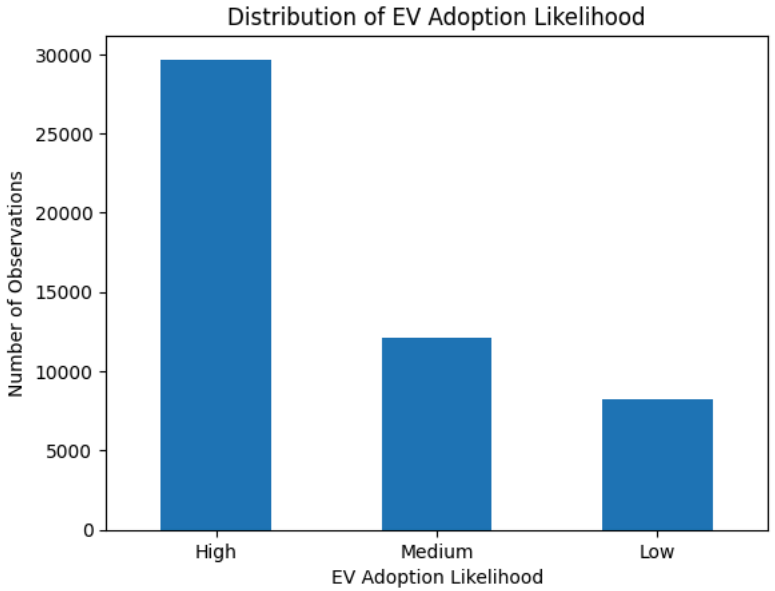
ev_adoption_likelihood	proportion
High	59.340
Medium	24.156
Low	16.504

dtype: float64

```
import matplotlib.pyplot as plt
```

```
df["ev_adoption_likelihood"].value_counts().plot(kind="bar")

plt.title("Distribution of EV Adoption Likelihood")
plt.xlabel("EV Adoption Likelihood")
plt.ylabel("Number of Observations")
plt.xticks(rotation=0)
plt.show()
```




```
df.describe().T
```

	count	mean	std	min	25%	50%	75%	max
age	50000.0	44.996080	14.110844	21.00	33.00	45.00	57.00	69.00
annual_income	50000.0	44311.869380	26109.170784	5104.00	25617.75	37891.00	56051.25	250000.00
daily_commute_km	50000.0	35.169358	14.736042	5.00	25.00	35.10	45.20	97.90
weekly_travel_distance_km	50000.0	228.727200	101.147254	25.00	157.70	224.10	294.50	732.30
vehicle_age_years	50000.0	6.012842	2.947132	0.00	3.90	6.00	8.00	20.00
fuel_expense_per_month	50000.0	295.169608	130.178552	-99.70	205.20	294.80	383.00	862.00
charging_station_accessibility	49500.0	5.892947	2.152186	1.00	4.40	5.90	7.40	10.00
nearest_charging_station_km	50000.0	7.132370	4.801992	0.50	3.00	6.90	10.70	28.50
home_charging_available	50000.0	0.649760	0.477050	0.00	0.00	1.00	1.00	1.00
electricity_cost_per_kwh	50000.0	0.215359	0.077914	0.08	0.15	0.22	0.28	0.35
environmental_awareness_score	50000.0	6.831854	1.920217	2.00	5.40	7.00	8.40	10.00
government_incentive_awareness	50000.0	5.979970	1.955106	1.00	4.60	6.00	7.30	10.00
technology_affinity_score	50000.0	6.721914	1.990730	2.00	5.20	6.70	8.50	10.00
range_anxiety_score	50000.0	5.288046	2.059862	2.00	3.70	5.00	6.80	10.00
battery_replacement_concern	50000.0	4.998338	1.955411	1.00	3.60	5.00	6.30	10.00
ev_knowledge_score	49500.0	6.755673	1.945842	1.00	5.30	7.00	8.30	10.00
previous_ev_experience	50000.0	0.198200	0.398648	0.00	0.00	0.00	0.00	1.00
monthly_energy_consumption_kwh	50000.0	186.740260	88.245302	15.60	124.50	179.30	241.50	645.90
monthly_charging_cost	50000.0	40.235028	24.940343	1.40	21.60	35.20	54.10	199.60

Data Error Correction and Imputation

Negative fuel expense: 271 rows have negative `fuel_expense_per_month` values, which cannot occur in reality. We treat this as a data entry error (likely a sign error) and correct it using absolute value rather than dropping the rows, since the magnitude still appears valid and 271 rows is a small fraction of the data (~0.5%).

Missing values: `education_level` is imputed with its mode (most common category), since it's categorical. `charging_station_accessibility` and `ev_knowledge_score` are imputed with their medians, since both are bounded 1–10 score variables where median is more robust to skew than mean.

```
(df["fuel_expense_per_month"] < 0).sum()
```

```
np.int64(271)
```

```
df[df["fuel_expense_per_month"] < 0][
    ["fuel_expense_per_month", "daily_commute_km", "current_vehicle_type"]
].head(10)
```

	fuel_expense_per_month	daily_commute_km	current_vehicle_type
18	-0.7	5.0	Sedan
27	-0.8	7.9	Truck
29	-19.2	5.0	Sedan
137	-11.9	6.0	Sedan
492	-4.5	5.7	Hatchback
688	-22.5	9.7	Sedan
746	-9.2	5.0	SUV
821	-17.5	5.1	Hatchback
874	-20.8	5.0	Sedan
1089	-8.4	5.0	Sedan

```
# Fix negative fuel_expense_per_month values using absolute values
df["fuel_expense_per_month"] = df["fuel_expense_per_month"].abs()

# verify removal of negative values
print((df["fuel_expense_per_month"] < 0).sum())

# Impute missing values
df["education_level"] = df["education_level"].fillna(df["education_level"].mode()[0])

df["charging_station_accessibility"] = df["charging_station_accessibility"].fillna(
    df["charging_station_accessibility"].median()
)

df["ev_knowledge_score"] = df["ev_knowledge_score"].fillna(
    df["ev_knowledge_score"].median()
)

# verify no missing values remain
print(df.isnull().sum().sum())
```

```
0
0
```

Outlier Detection

Using the IQR method (1.5×IQR beyond Q1/Q3) across the remaining numeric columns, outlier rates range from 0% (`age`, `electricity_cost_per_kwh`) up to ~3.8% (`annual_income`). All flagged outliers are right-tail only (no implausible lower-bound values), consistent with natural right-skew in variables like income and travel distance rather than data errors. We retain these values rather than removing or capping them, since they represent plausible real-world cases. We note `annual_income` and `monthly_charging_cost` as candidates for log-transformation if a linear/distance-based model proves sensitive to their skew.

```
import numpy as np

# Numeric columns to check for outliers
```

```
cols_to_check = [
    "age", "annual_income", "daily_commute_km", "weekly_travel_distance_km",
    "vehicle_age_years", "fuel_expense_per_month", "nearest_charging_station_km",
    "electricity_cost_per_kwh", "monthly_energy_consumption_kwh", "monthly_charging_cost"
]

# IQR method - flag values outside 1.5*IQR from Q1/Q3
outlier_summary = {}

for col in cols_to_check:
    Q1 = df[col].quantile(0.25)
    Q3 = df[col].quantile(0.75)
    IQR = Q3 - Q1
    lower = Q1 - 1.5 * IQR
    upper = Q3 + 1.5 * IQR
    outliers = df[(df[col] < lower) | (df[col] > upper)]
    outlier_summary[col] = len(outliers)

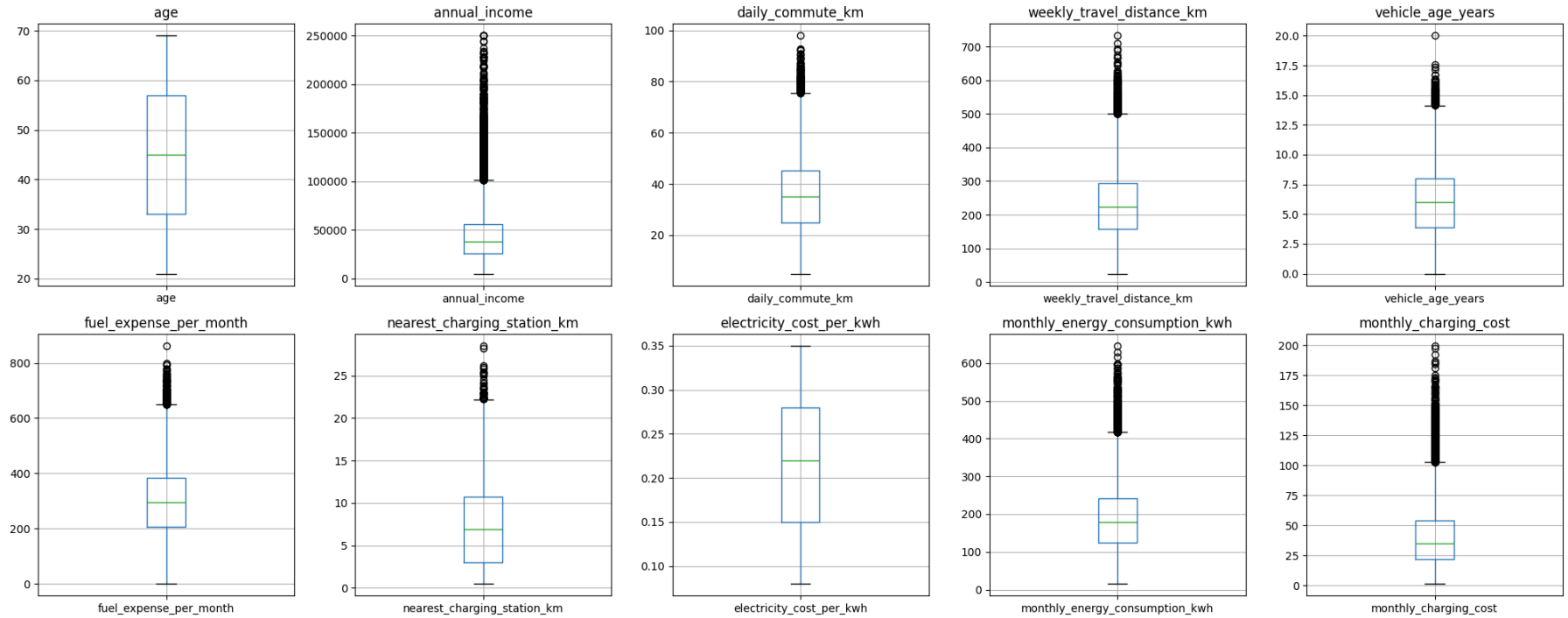
outlier_df = pd.DataFrame.from_dict(outlier_summary, orient="index", columns=["outlier_count"])
outlier_df["outlier_pct"] = (outlier_df["outlier_count"] / len(df)) * 100
print(outlier_df.sort_values("outlier_pct", ascending=False))
```

```
                               outlier_count  outlier_pct
annual_income                           1882        3.764
monthly_charging_cost                   1124        2.248
monthly_energy_consumption_kwh           646        1.292
weekly_travel_distance_km                377        0.754
daily_commute_km                         192        0.384
fuel_expense_per_month                   173        0.346
vehicle_age_years                        170        0.340
nearest_charging_station_km               42        0.084
age                                        0        0.000
electricity_cost_per_kwh                   0        0.000
```

```
# Boxplots
fig, axes = plt.subplots(2, 5, figsize=(20, 8))
axes = axes.flatten()

for i, col in enumerate(cols_to_check):
    df.boxplot(column=col, ax=axes[i])
    axes[i].set_title(col)

plt.tight_layout()
plt.show()
```



Correlation Analysis

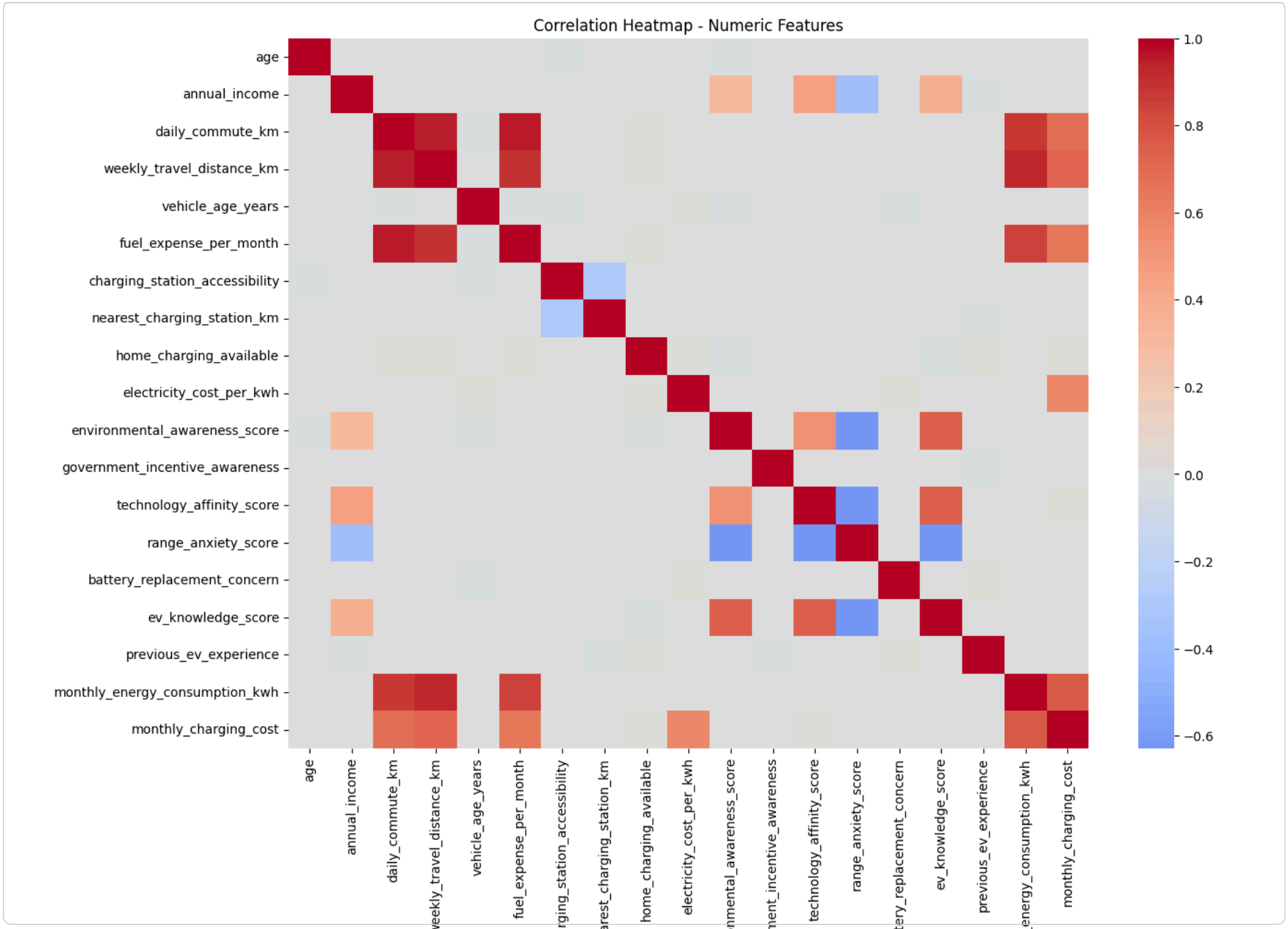
A correlation heatmap across numeric features reveals a multicollinear cluster: `daily_commute_km`, `weekly_travel_distance_km`, `fuel_expense_per_month`, `monthly_energy_consumption_kwh`, and `monthly_charging_cost` are all correlated at 0.7–0.95, which makes intuitive sense (more driving drives up fuel/energy/charging costs together). We address this below by dropping two redundant columns.

```
# Correlation heatmap (numeric features only)
import seaborn as sns

numeric_cols = df.select_dtypes(include=[np.number]).columns
corr = df[numeric_cols].corr()
```

```
plt.figure(figsize=(14, 10))
sns.heatmap(corr, cmap="coolwarm", center=0, annot=False)
plt.title("Correlation Heatmap - Numeric Features")
plt.show()

# Top 10 Correlated features
corr_pairs = corr.abs().unstack().sort_values(ascending=False)
corr_pairs = corr_pairs[corr_pairs < 1].drop_duplicates()
print(corr_pairs.head(10))
```



Feature Relationships with Target

```
daily_commute_km              fuel_expense_per_month        0.951789
weekly_travel_distance_km     daily_commute_km              0.944852
```

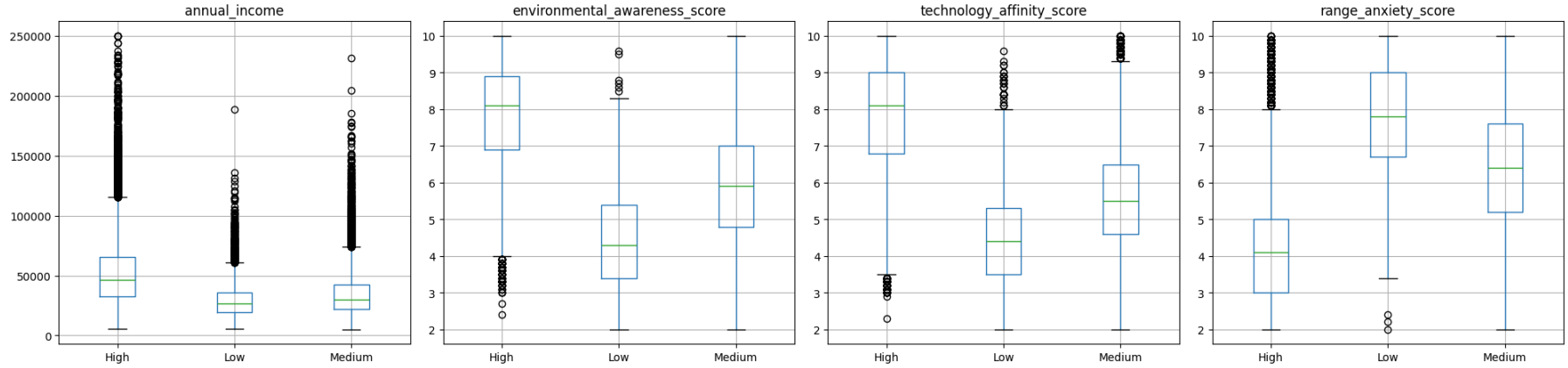
```
monthly_energy_consumption_kwh  weekly_travel_distance_km      0.842395
weekly_travel_distance_km       fuel_expense_per_month         0.765091
fuel_expense_per_month          monthly_energy_consumption_kwh 0.752686
monthly_energy_consumption_kwh  monthly_charging_cost          0.752686
ev_knowledge_score              technology_affinity_score      0.752686
```

Grouping key features by adoption class shows clean separation for `environmental_awareness_score`, `technology_affinity_score`, and `range_anxiety_score` — High-adoption respondents show higher awareness/tech affinity and lower range anxiety than Low-adoption respondents. These are expected to be strong predictors for modeling.

```
# Boxplots of a few key features grouped by target class
features_to_compare = ["annual_income", "environmental_awareness_score",
                       "technology_affinity_score", "range_anxiety_score"]

fig, axes = plt.subplots(1, 4, figsize=(20, 5))
for i, col in enumerate(features_to_compare):
    df.boxplot(column=col, by="ev_adoption_likelihood", ax=axes[i])
    axes[i].set_title(col)
    axes[i].set_xlabel("")

plt.suptitle("")
plt.tight_layout()
plt.show()
```



Feature Selection: Resolving Multicollinearity

To reduce redundancy identified in the correlation analysis, we drop `daily_commute_km` (redundant with `weekly_travel_distance_km`, which captures the same usage pattern at a less noisy weekly granularity) and `monthly_energy_consumption_kwh` (redundant with `monthly_charging_cost`, the more directly interpretable variable). This reduces multicollinearity for linear models without losing meaningful insight.

```
# Drop redundant/multicollinear columns
df = df.drop(columns=["daily_commute_km", "monthly_energy_consumption_kwh"])
```

```
print(df.shape)
```

```
(50000, 21)
```

Encoding

```
print(df.shape)
```

```
(50000, 21)
```

Encoding